

[ASRJC/2025/Q2]

2 Name your Jupyter Notebook as:

TASK2_<your name>_<centre number>_<index number>.ipynb

A marathon organiser needs a system to manage registration data. Based on past years' participation data, the organiser is expecting 15,000 runners for this year's event. The system must handle runner registration data efficiently.

For each of the sub-tasks, add a comment statement at the beginning of the code using the symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 2.1
        Program code
```

Output:

Task 2.1

Write a function `generate_runners()` to:

- generate 15 000 **distinct** random 8-digit handphone numbers ranging from 8000 0000 to 9899 9999 (both inclusive)
- pair each phone number with a randomly selected race category from Men's Marathon, Women's Marathon, Men's Half-Marathon, Women's Half-Marathon to form a tuple
- store all tuples in a list and save the data in the list to a text file `runners.txt`
- return the list (containing 15 000 tuples)

For example, the first 2 lines of `runners.txt` might look like this:

```
87120982,Women's Marathon
91876543,Men's Half-Marathon
```

Test your function, storing the returned list in a variable named `runners`.
You do not need to output the list in the notebook.

[4]

Task 2.2

Write program code to:

- implement a **merge sort** algorithm to sort the data based on phone numbers in ascending order (you are **not** to use a built-in sorting function) and store the sorted list as `sorted_data`
- use the `timeit` module to measure and output the time taken for the merge sort operation (refer to `sample_code.py` for sample implementation)
- store the sorted data (phone number, category) in a text file `runners_sorted1.txt`.

Test the program and show the time taken for merge sort.

[7]

[ASRJC/2025/Q2 Cont']

Task 2.3

Write program code to:

- create a `Node` class and an `OrderedLL` class to implement an **ordered** linked list
- define the following properties in the `Node` class: `phone`, `category`, `next`
- define the following methods in the `OrderedLL` class:
 - `insert(phone, category)`: insert a new runner while maintaining phone number order
 - `delete(phone)`: remove a runner by phone number, returns `True` if a runner is removed, and returns `False` if no runner is identified for removal
 - `update(phone, new_category)`: update the race category for a given phone number
 - `get_list()`: return a list of all tuples in traversal order
- use the `timeit` module to measure and output the overall time taken to populate the ordered linked list
- retrieve the sorted data using `get_list()` and write to a text file `runners_sorted2.txt`. [10]

Task 2.4

Write program code to implement a **recursive** binary search function to search for a runner by phone number.

The function should take the sorted list and a phone number as parameters, and return the runner's category if found, or "Runner not found." if the phone number does not exist in the list.

Test the function by searching for at least 3 different phone numbers in `sorted_data`: one at the end of the sorted list, one near the middle of the sorted list, and one that does not exist in the list. [4]

Task 2.5

On race pack collection day, the marathon organiser has support crew members responsible for packing race bibs and t-shirts into race packs. For efficient operations, when runners arrive at the entrance of the collection venue, they will scan their personalised QR codes. Their runner data will be added to a queue for the crew members to process. These crew members then prepare the race packs according to the queue sequence and deliver them to the collection counter, allowing runners to receive their race packs promptly without extended waiting times.

Write program code to implement a **circular queue** class `CQueue` that uses a 1D array of length 5 to store the data, as well as a front pointer and a rear pointer to track the array indices.

Include the following methods:

- `is_full()`: check if the queue is fully occupied
- `is_empty()`: check if the queue is empty
- `enqueue(phone, category)`: add runner data to the queue
- `dequeue()`: remove and return the runner data
- `display()`: print the current state of the queue, from the front to the rear

Create a queue `rq` and test the key methods.

[7]

Save your Jupyter Notebook for Task 2.